Parallel & Distributed Computing

UCS645

Assignment-1

Name: Kartik Juneja

Roll No: 102303274

System Environment: Linux(Fedora), GCC compiler

AIM : To learn basics of OpenMP

Q1: DAXPY Loop

CODE:

```
#include<stdio.h>
#include<omp.h>
#include<stdlib.h>
#define N 65536 //coz x and y are vectors of size 2^16
int main(){
    int a  = 2;
    double* x = (double*)malloc(N * sizeof(double));
    double* y = (double*)malloc(N * sizeof(double));
    double p1_start, p1_end, t_p1, pi_start, pi_end, t_pi;
    double speedup;
    //one thread
    p1_start = omp_get_wtime();
    for(int i = 0; i < N; i++){
        x[i] = a * x[i] + y[i];
    }
    p1_end = omp_get_wtime();
```

```
    t_p1 = p1_end - p1_start;
    speedup = t_p1 / t_p1;
    printf("1 Thread: Time = %f, SpeedUp = %f\n", t_p1, speedup);
    //n threads
    int maxThreads = omp_get_max_threads();
    for(int n = 2; n <= maxThreads; n++){
        omp_set_num_threads(n);
        pi_start = omp_get_wtime();
        #pragma omp parallel for
        for(int i = 0; i < N; i++){
            x[i] = a * x[i] + y[i];
        }
        pi_end = omp_get_wtime();
        t_pi = pi_end - pi_start;
        speedup = t_p1 / t_pi;
        printf("%d Thread: Time = %f, SpeedUp = %f\n", n, t_pi, speedup);
    }
    free(x);
    free(y);
    return 0;
}
```

OUTPUT:

```
1 Thread: Time = 0.000935, SpeedUp = 1.000000
2 Thread: Time = 0.000435, SpeedUp = 2.150758
3 Thread: Time = 0.000339, SpeedUp = 2.761146
4 Thread: Time = 0.000186, SpeedUp = 5.034554
5 Thread: Time = 0.000206, SpeedUp = 4.548376
6 Thread: Time = 0.000194, SpeedUp = 4.821900
7 Thread: Time = 0.000190, SpeedUp = 4.913874
8 Thread: Time = 0.000171, SpeedUp = 5.480169
9 Thread: Time = 0.000176, SpeedUp = 5.304913
10 Thread: Time = 0.000186, SpeedUp = 5.018186
11 Thread: Time = 0.000213, SpeedUp = 4.390380
12 Thread: Time = 0.000233, SpeedUp = 4.021003
13 Thread: Time = 0.000270, SpeedUp = 3.461751
14 Thread: Time = 0.000264, SpeedUp = 3.541194
15 Thread: Time = 0.000220, SpeedUp = 4.253565
16 Thread: Time = 0.000234, SpeedUp = 4.003124
17 Thread: Time = 0.000207, SpeedUp = 4.518583
18 Thread: Time = 0.000248, SpeedUp = 3.773306
19 Thread: Time = 0.000219, SpeedUp = 4.276062
20 Thread: Time = 0.000216, SpeedUp = 4.328257
```

OBSERVATION:

Maximum speedup of 5.480169 is achieved using **8 threads**. Increasing the thread count after this continues to make the speedup fluctuate a lot ranging all the way between approx 3.5 to 5.3. This is because all these threads are trying to access the memory bus at the same time thus after a certain number of threads the cost of synchronizing these threads exceeds the benefit of parallelism.

Also because DAXPY loop is a memory-bound process, the efficiency bottleneck is the memory access speeds and not the number of CPU cores.

OBSERVATION FROM PERF:

```
Performance counter stats for './outputs/q1':

              400      context-switches                 #  24458.7 cs/sec  cs_per_second
               49      cpu-migrations                   #   2996.2 migrations/sec  migrations_per_second
              505      page-faults                      #  30879.1 faults/sec  page_faults_per_second
            16.35 msec task-clock                       #      1.9 CPUs  CPUs_utilized
           37,411      cpu_core/branch-misses/          #      1.5 %  branch_miss_rate         (75.82%)
        2,541,395      cpu_core/branches/               #    155.4 M/sec  branch_frequency     (75.82%)
       14,658,710      cpu_core/cpu-cycles/             #      0.9 GHz  cycles_frequency       (75.82%)
       32,030,286      cpu_core/instructions/           #      2.2 instructions  insn_per_cycle  (75.82%)
           22,654      cpu_atom/branch-misses/          #      1.3 %  branch_miss_rate         (52.00%)
        1,849,765      cpu_atom/branches/               #    113.1 M/sec  branch_frequency     (53.16%)
       12,852,547      cpu_atom/cpu-cycles/             #      0.8 GHz  cycles_frequency       (53.16%)
       30,380,087      cpu_atom/instructions/           #      2.4 instructions  insn_per_cycle  (53.16%)
            TopdownL1 (cpu_core)                        #      5.6 %  tma_bad_speculation
                                                        #     28.2 %  tma_frontend_bound       (75.82%)
                                                        #     22.3 %  tma_backend_bound
                                                        #     43.8 %  tma_retiring             (75.82%)
            TopdownL1 (cpu_atom)                        #     13.1 %  tma_backend_bound        (5.27%)
                                                        #      nan %  tma_frontend_bound       (0.00%)
                                                        #      nan %  tma_bad_speculation
                                                        #      nan %  tma_retiring             (0.00%)

      0.007668348 seconds time elapsed

      0.001328000 seconds user
      0.020520000 seconds sys
```

The metric **1.9 CPUs_utilized** indicates that some level of parallelism is achieved and the CPU is executing **2.2 instructions per cycle** which is moderate for a modern Inter CPU.

Q2. Matrix Multiply

CODE:

```
#include <stdio.h>
#include <omp.h>
#define SIZE 1000
double A[SIZE][SIZE], B[SIZE][SIZE], C[SIZE][SIZE];
int main() {
    for(int i=0; i<SIZE; i++) for(int j=0; j<SIZE; j++) { A[i][j]=1.0; B[i][j]=2.0; }
    double start, end, seq_time;
    start = omp_get_wtime();
    for (int i=0; i<SIZE; i++)
        for (int j=0; j<SIZE; j++) {
            double sum = 0;
            for (int k=0; k<SIZE; k++) sum += A[i][k] * B[k][j];
            C[i][j] = sum;
        }
    end = omp_get_wtime();
    seq_time = end - start;
    printf("Sequential Time: %f sec\n", seq_time);
    start = omp_get_wtime();
    #pragma omp parallel for
    for (int i=0; i<SIZE; i++)
        for (int j=0; j<SIZE; j++) {
            double sum = 0;
            for (int k=0; k<SIZE; k++) sum += A[i][k] * B[k][j];
            C[i][j] = sum;
        }
    printf("1D Parallel Time: %f sec (Speedup: %fx)\n", omp_get_wtime()-start,
seq_time/(omp_get_wtime()-start));
```

```
    start = omp_get_wtime();
    #pragma omp parallel for
    for (int i=0; i<SIZE; i++)
        #pragma omp parallel for
        for (int j=0; j<SIZE; j++) {
            double sum = 0;
            for (int k=0; k<SIZE; k++) sum += A[i][k] * B[k][j];
            C[i][j] = sum;
        }
    printf("2D Parallel Time: %f sec (Speedup: %fx)\n", omp_get_wtime()-start,
seq_time/(omp_get_wtime()-start));
    start = omp_get_wtime();
    #pragma omp parallel for collapse(2)
    for (int i=0; i<SIZE; i++)
        for (int j=0; j<SIZE; j++) {
            double sum = 0;
            for (int k=0; k<SIZE; k++) sum += A[i][k] * B[k][j];
            C[i][j] = sum;
        }
    printf("2D (collapsed) Parallel Time: %f sec (Speedup: %fx)\n", omp_get_wtime()-start,
seq_time/(omp_get_wtime()-start));
    return 0;
}
```

OUTPUT:

```
Sequential Time: 3.590269 sec
1D Parallel Time: 0.589309 sec (Speedup: 6.092340x)
2D Parallel Time: 0.574793 sec (Speedup: 6.246197x)
2D (collapsed) Parallel Time: 0.573856 sec (Speedup: 6.256402x)
```

OBSERVATION:

The implementation tests three OpenMP parallelization strategies for matrix multiplication: parallelizing only the outer loop (1D), using nested parallel directives (2D), and using the collapse(2) clause (2D collapsed). The sequential baseline ran at 3.590 seconds. Naive 2D parallelism offered minimal improvement over 1D because OpenMP discourages nested parallel regions and tends to serialize the inner loop. The collapse(2) clause merges both the i and j loops into a single flat iteration space of 1,000,000 iterations, giving the scheduler far more work to distribute evenly across threads, which is why it achieved the best speedup of ~6.26× compared to ~6.09× for 1D and ~6.25× for naive 2D.

OBSERVATION FROM PERF:

```
Performance counter stats for './outputs/q2':

            1,153      context-switches                 #     31.7 cs/sec  cs_per_second
               69      cpu-migrations                   #      1.9 migrations/sec  migrations_per_second
            5,990      page-faults                      #    164.7 faults/sec  page_faults_per_second
        36,359.52 msec task-clock                       #      6.8 CPUs  CPUs_utilized
        3,520,409      cpu_core/branch-misses/          #      0.1 %  branch_miss_rate         (85.60%)
    5,392,176,889      cpu_core/branches/               #    148.3 M/sec  branch_frequency     (85.60%)
   36,248,804,937      cpu_core/cpu-cycles/             #      1.0 GHz  cycles_frequency       (85.60%)
   70,738,469,221      cpu_core/instructions/           #      2.0 instructions  insn_per_cycle  (85.60%)
        2,988,835      cpu_atom/branch-misses/          #      0.1 %  branch_miss_rate         (29.74%)
    4,935,339,246      cpu_atom/branches/               #    135.7 M/sec  branch_frequency     (29.71%)
   30,836,397,543      cpu_atom/cpu-cycles/             #      0.8 GHz  cycles_frequency       (29.11%)
   55,098,468,407      cpu_atom/instructions/           #      1.8 instructions  insn_per_cycle  (29.15%)
            TopdownL1 (cpu_core)                        #      2.0 %  tma_bad_speculation
                                                        #     12.2 %  tma_frontend_bound       (85.60%)
                                                        #     32.8 %  tma_backend_bound
                                                        #     53.1 %  tma_retiring             (85.60%)
            TopdownL1 (cpu_atom)                        #     56.6 %  tma_backend_bound        (34.02%)
                                                        #      5.2 %  tma_frontend_bound       (34.03%)
                                                        #      2.4 %  tma_bad_speculation
                                                        #     35.8 %  tma_retiring             (34.02%)

      5.346153472 seconds time elapsed

     35.916006000 seconds user
      0.055323000 seconds sys
```

The perf stats show that the program actively utilized around 6.8 CPU cores on average, confirming genuine multi-threaded execution. The branch miss rate was extremely low at just 0.1%, which is expected since the triple-loop structure of matrix multiplication follows a fixed, predictable pattern. The IPC of 2.0 on the cpu_core indicates decent pipeline efficiency, though the TopdownL1 analysis flags 32.8% of cycles as backend-bound, meaning the program spent a significant portion of time waiting on memory — likely due to cache misses when accessing column elements of matrix B in the inner loop. Overall, the bottleneck is memory latency rather than compute or branch mispredictions.

Q3. Pi Calculation

CODE:

```
#include <stdio.h>
#include <omp.h>
static long num_steps = 100000000;
int main() {
    double pi, sum = 0.0;
    double step = 1.0 / (double)num_steps;
    double start, end, seq_time;
    start = omp_get_wtime();
    for (int i = 0; i < num_steps; i++) {
        double x = (i + 0.5) * step;
        sum += 4.0 / (1.0 + x * x);
    }
    pi = step * sum;
    end = omp_get_wtime();
    seq_time = end - start;
    printf("Sequential Pi: %.10f, Time: %f sec\n", pi, seq_time);
    sum = 0.0;
    start = omp_get_wtime();
    #pragma omp parallel for reduction(+:sum)
    for (int i = 0; i < num_steps; i++) {
        double x = (i + 0.5) * step;
        sum += 4.0 / (1.0 + x * x);
    }
    pi = step * sum;
    end = omp_get_wtime();
    printf("Parallel Pi:   %.10f, Time: %f sec\n", pi, end - start);
    printf("Speedup: %fx\n", seq_time / (end - start));
```

```
    return 0;
}
```

OUTPUT:

```
Sequential Pi: 3.1415926536, Time: 0.357164 sec
Parallel Pi:   3.1415926536, Time: 0.064823 sec
Speedup: 5.509816x
```

OBSERVATION:

The implementation estimates Pi using numerical integration over 100 million steps. The parallel version uses OpenMP's reduction(+:sum) clause, which allows each thread to maintain its own private partial sum and safely combine them at the end, avoiding race conditions without needing locks. The thread count is left to the OpenMP runtime to decide based on available cores, ensuring efficient utilization. The sequential version ran at 0.357 seconds while the parallel version completed in 0.064 seconds, achieving a speedup of ~5.51×.

OBSERVATION FROM PERF:

```
Performance counter stats for './outputs/q3':

              135      context-switches                 #    110.9 cs/sec  cs_per_second
               25      cpu-migrations                   #     20.5 migrations/sec  migrations_per_second
              116      page-faults                      #     95.3 faults/sec  page_faults_per_second
        1,216.96 msec task-clock                        #      2.9 CPUs  CPUs_utilized
           30,417      cpu_core/branch-misses/          #      0.0 %  branch_miss_rate         (86.66%)
      188,848,021      cpu_core/branches/               #    155.2 M/sec  branch_frequency     (86.66%)
    1,663,398,666      cpu_core/cpu-cycles/             #      1.4 GHz  cycles_frequency       (86.66%)
    4,018,308,807      cpu_core/instructions/           #      2.4 instructions  insn_per_cycle  (86.66%)
           50,745      cpu_atom/branch-misses/          #      0.1 %  branch_miss_rate         (28.55%)
       78,983,677      cpu_atom/branches/               #     64.9 M/sec  branch_frequency     (29.23%)
      934,849,333      cpu_atom/cpu-cycles/             #      0.8 GHz  cycles_frequency       (29.33%)
    1,462,756,886      cpu_atom/instructions/           #      1.6 instructions  insn_per_cycle  (29.39%)
            TopdownL1 (cpu_core)                        #      0.8 %  tma_bad_speculation
                                                        #      7.4 %  tma_frontend_bound       (86.66%)
                                                        #     40.8 %  tma_backend_bound
                                                        #     51.0 %  tma_retiring             (86.66%)
            TopdownL1 (cpu_atom)                        #     68.2 %  tma_backend_bound        (34.44%)
                                                        #      0.5 %  tma_frontend_bound       (34.13%)
                                                        #     -0.3 %  tma_bad_speculation
                                                        #     31.7 %  tma_retiring             (33.99%)

      0.424534144 seconds time elapsed

      1.210271000 seconds user
      0.003975000 seconds sys
```

The program utilized 2.9 CPUs on average, reflecting reasonable thread utilization given the short runtime of the workload. The IPC of 2.4 on cpu_core indicates efficient pipeline usage for the arithmetic-heavy division and addition in each step, and the branch miss rate is effectively 0%, consistent with the simple unconditional loop structure. The TopdownL1 analysis shows 40.8% backend-bound on cpu_core and 68.2% on cpu_atom, meaning execution unit or memory stalls are the primary bottleneck, though the 51% retiring rate on cpu_core confirms that a good portion of cycles are still being spent doing useful computation.